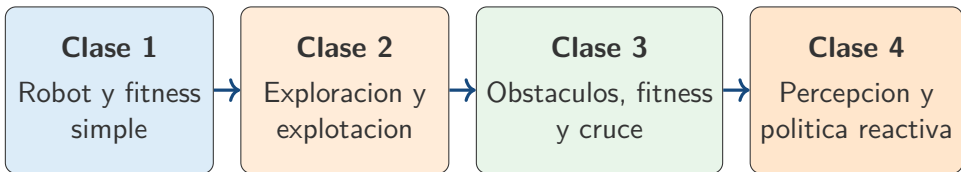


Percepcion local

De evolucionar rutas a evolucionar politicas reactivas

Laboratorio de Inteligencia Artificial



La pregunta nueva

¿Puede evolucionar una regla de comportamiento que consulte el entorno mientras se mueve?

Clase 3: open loop

D D D L D R R R ...

El ADN era una ruta completa. El robot recibia el plan antes de empezar y no lo modificaba durante el recorrido.

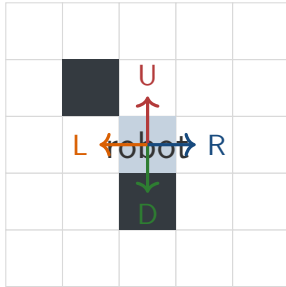
Clase 4: closed loop

observacion $\rightarrow$ regla $\rightarrow$ accion

El ADN es una politica. La siguiente accion depende de lo que el robot observa en ese momento.

Cambio conceptual

Ya no evolucionamos una secuencia para un recorrido; evolucionamos reglas reutilizables para responder a situaciones.



Observa cuatro celdas vecinas en orden:

U D L R

- 0: celda libre.
- 1: obstaculo o borde.
- No observa el tablero completo.

Percepcion parcial

El robot conoce solo una pequena parte del estado del entorno.

$2^4 = 16$ patrones de sensores

Codificación

Los bits se leen como U D L R y forman un índice entre 0 y 15.

0000 $\rightarrow$ 0

0001 $\rightarrow$ 1

1010 $\rightarrow$ 10

1111 $\rightarrow$ 15

ADN de la política

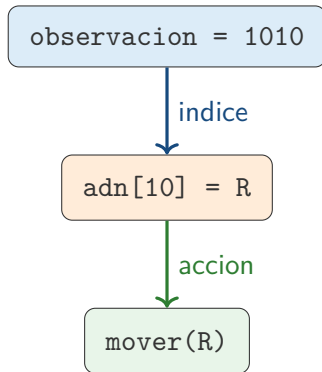
El ADN tiene 16 genes. Cada gen contiene una acción:

$\text{adn}[\text{observacion}] \rightarrow \text{U, D, L o R}$

Una misma regla puede utilizarse muchas veces durante un recorrido.

Representación

En la clase 3: 18 genes = 18 pasos. En la clase 4: 16 genes = 16 situaciones posibles.



si cambia la observacion,
puede cambiar la accion

Bucle de control

Observar → consultar la politica → intentar mover → volver a observar.

Problema y evaluacion

- Misma cuadrícula, inicio y meta.
- Entorno con obstaculos.
- Mismo fitness.
- Mismos intentos bloqueados y metricas.

Busqueda evolutiva

- Misma poblacion.
- Elitismo.
- Mutacion.
- Cruce de un punto.
- Comparacion de variantes.

Solo cambia una pieza central

Cambia la representacion del individuo y la funcion que ejecuta su ADN.

$$\text{fitness} = 500 - 35d + 4p - 30c - 3r + 2000m$$

d

Distancia
a la meta

p

Pasos
utiles

c

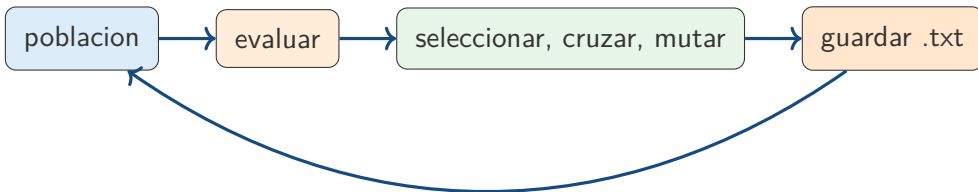
Choques
contra bloqueos

r, m

Repeticiones
y llegada

Nueva limitacion

Dos posiciones distintas pueden producir la misma observacion local. Sin memoria o direccion de la meta, una politica puede repetir el mismo comportamiento y entrar en un ciclo.



Persistencia experimental

La ultima generacion se guarda como generacionN_identificador.txt. El archivo contiene toda la poblacion, no solo el mejor individuo, con sus reglas observacion -> accion.

`--identificador experimento_a`

```
python3 robot_percepcion.py --modo mutacion --semilla 7 --sin-pausa --identificador experimento_a
archivo=$(ls -t generacion*\_experimento_a.txt | head -n 1)
python3 robot_percepcion.py --cargar "$archivo" --ejecutar mejor --sin-pausa
```

Dos formas de observar la generacion

--ejecutar mejor: anima la politica con mayor fitness.

--ejecutar todos: recorre las politicas guardadas sobre el mismo mapa.

Reproducibilidad

Podemos comparar, explicar y volver a ejecutar una generacion sin evolucionarla de nuevo.

- 1 Ejecuta el modo comparativo con las semillas 7, 21 y 42.
- 2 Registra llegada, generaciones, puntaje, choques, pasos utiles y diversidad.
- 3 Guarda cada experimento con un identificador distinto.
- 4 Carga la generacion y ejecuta primero el mejor; despues, todos.

```
python3 robot_percepcion.py --semilla 7 --sin-pausa
```

Pregunta

¿La politica que obtiene mejor fitness es tambien la que se comporta mejor al observar su trayectoria?

Realiza las cuatro modificaciones y conserva las demas condiciones del experimento:

- ➊ Agrega la direccion relativa de la meta a la observacion.
- ➋ Incorpora una memoria de la ultima accion.
- ➌ Cambia el numero maximo de pasos y explica los ciclos observados.
- ➍ Compara la misma politica en una segunda distribucion de obstaculos.

Importante

No son alternativas. Cada modificacion debe tener una prueba, datos y una explicacion de su efecto.

Una politica no memoriza una ruta: responde a situaciones.

Gana

Adaptacion durante la ejecucion y reglas reutilizables.

Pierde

Informacion global: puede confundir posiciones y repetir ciclos.

Siguiente paso

Mas sensores, memoria y pruebas en otros mapas.

Idea final

El algoritmo evolutivo sigue siendo el mismo; al cambiar la representacion, cambia la clase de conducta que puede aprender.

Politica actual

La observacion de cuatro bits selecciona directamente una regla:

$$\text{observacion} \rightarrow \text{adn}[\text{indice}] \rightarrow \text{accion}$$

Es una politica discreta: cada patron tiene una accion asignada.

Politica neuronal

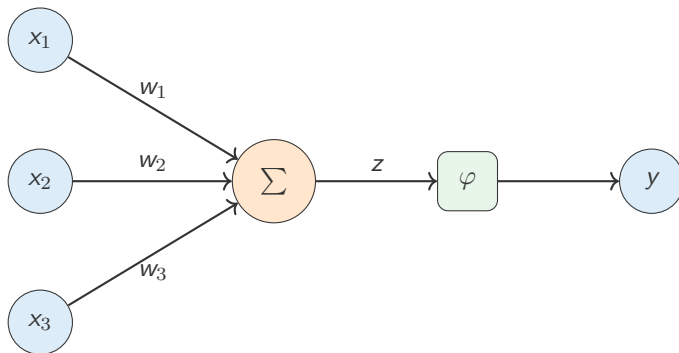
La observacion entra en una funcion parametrizada:

$$\text{observacion} \rightarrow \text{red} \rightarrow \text{accion}$$

Los pesos determinan como se combinan las entradas.

Idea de continuidad

En ambos casos queremos aproximar una politica: una funcion que transforma una observacion en una decision.



Operacion basica

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b \quad y = \varphi(z)$$

Los pesos w indican la influencia de cada entrada; b desplaza la suma; la activacion transforma el resultado.

ReLU

$$\varphi(z) = \max(0, z)$$

Simple y frecuente en capas ocultas. Conserva valores positivos y anula los negativos.

Sigmoide

$$\varphi(z) = \frac{1}{1+e^{-z}}$$

Produce valores entre 0 y 1.
Puede representar una probabilidad.

tanh

$$\varphi(z) = \tanh(z)$$

Produce valores entre -1 y 1.
Centra la salida alrededor de cero.

Sin activacion

Varias capas lineales seguirian siendo una sola transformacion lineal. La activacion permite representar relaciones no lineales.

Observacion del robot

Una lectura de sensores puede escribirse como:

$$\mathbf{x} = [x_U \quad x_D \quad x_L \quad x_R]^T$$

Por ejemplo, 1010 se convierte en $\mathbf{x} = [1, 0, 1, 0]^T$.

Transformacion de una capa

Muchas neuronas calculan en paralelo:

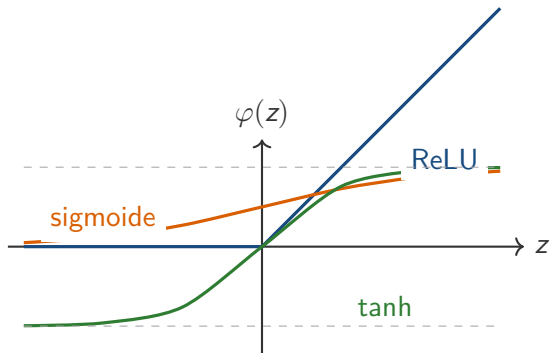
$$\mathbf{z} = W\mathbf{x} + \mathbf{b}$$

$$\mathbf{h} = \varphi(\mathbf{z})$$

W contiene los pesos y $\mathbf{b}$ los sesgos.

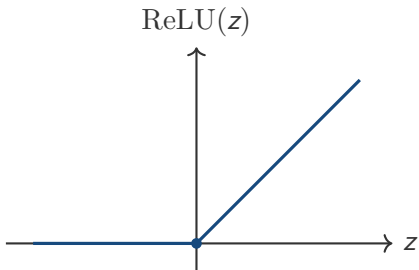
Lectura intuitiva

La capa no memoriza solo un caso: combina las entradas y puede aprender patrones de varias observaciones a la vez.



Tres comportamientos

ReLU no satura en la zona positiva; sigmoide comprime a $[0, 1]$; tanh comprime a $[-1, 1]$.

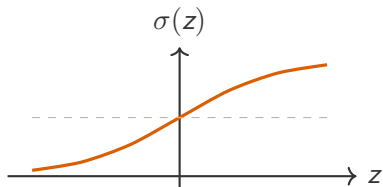


Definicion

$$\text{ReLU}(z) = \max(0, z)$$

Si la suma es negativa, la salida es cero. Si es positiva, la salida crece linealmente.

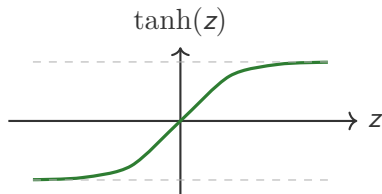
- Es rapida de calcular.
- Es una eleccion habitual en capas ocultas.
- Introduce una no linealidad con una regla sencilla.



Sigmoide

$$\sigma(z) = 1/(1 + e^{-z})$$

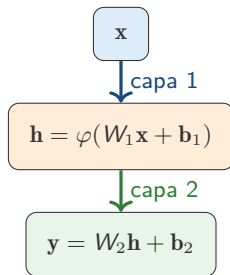
Interpretable como intensidad o probabilidad.



tanh

$$\tanh(z)$$

Centrada en cero, util cuando las senales positivas y negativas deben equilibrarse.



Composicion

La salida de una capa se convierte en la entrada de la siguiente. La capa oculta transforma la representacion y la salida la convierte en decisiones.

Por que varias capas

Cada capa puede detectar combinaciones mas complejas: primero patrones simples y despues combinaciones de esos patrones.

Cuatro salidas

La ultima capa puede producir cuatro valores:

$$[o_U, o_D, o_L, o_R]$$

La accion elegida es la salida mayor:

$$\text{accion} = \arg \max(o_U, o_D, o_L, o_R)$$

Ejemplo

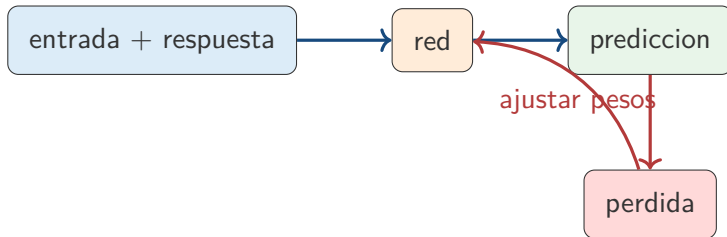
$$[0.2, 0.8, 0.1, 0.4]$$

D

La red prefiere bajar porque o_D es el mayor valor.

Otra opcion

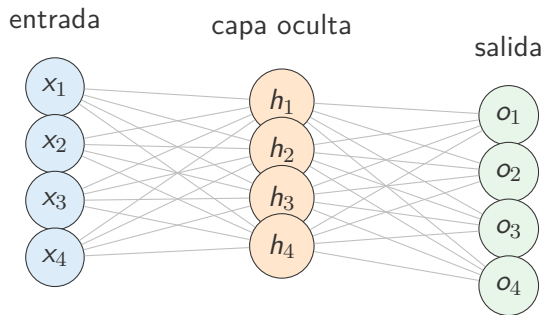
Una salida con softmax puede convertirse en probabilidades y permitir decisiones estocasticas, no solo elegir siempre el maximo.



Idea basica

La perdida mide cuanto se aleja la prediccion de la respuesta esperada. El gradiente indica en que direccion cambiar los pesos para reducirla.

En una red entrenada con ejemplos, este ciclo se repite muchas veces: propagacion hacia delante, calculo de perdida y retropropagacion.



- **Feed-forward:** la informacion avanza hacia delante, sin ciclos internos.
- **Fully connected:** cada neurona de una capa se conecta con todas las de la siguiente.
- Cada conexion tiene un peso y cada neurona tiene un sesgo.

sensores $\rightarrow$ capa oculta $\rightarrow$ salidas

En cada paso del robot

- 1 Convertimos la observacion en entradas numericas, por ejemplo 0 1 0 1.
- 2 Cada capa calcula una suma ponderada y aplica una activacion.
- 3 La salida con mayor valor puede representar la accion elegida: U, D, L o R.

Importante

La red no ejecuta una ruta completa. Igual que nuestra politica local, recibe una observacion y decide una accion en cada instante.

Aprendizaje supervisado

Tenemos ejemplos de entrada y respuesta esperada. La red calcula un error y ajusta sus pesos mediante el gradiente y la retropropagación.

predicción → error → pesos nuevos

Nuestra evolución

No usamos etiquetas de acciones correctas. Evaluamos la conducta completa con fitness y seleccionamos las políticas que funcionan mejor.

política → recorrido → fitness

Dos formas de buscar parámetros

Los pesos de una red también pueden evolucionarse genéticamente. En ese caso, la red reemplaza la tabla de reglas, pero el fitness puede conservarse.

La pregunta sigue siendo la misma: que conducta queremos favorecer?

Tabla de reglas	Red neuronal	Problema comun
16 acciones discretas	Pesos y activaciones	Observacion → accion
Facil de inspeccionar	Mas flexible	Fitness y ciclos
Pocas reglas	Muchos parametros	Probar en otros mapas

Siguiente laboratorio

Construir una politica neuronal pequena y comparar sus decisiones con la politica evolutiva basada en reglas.